PSScript Project Improvements Roadmap

10 High-Impact Improvements for 2026

Based on current best practices, performance analysis, and codebase review.

Table of Contents

1. Replace Babel with SWC for Faster Builds
2. Implement Comprehensive Test Coverage
3. Add Real AI Provider Integration
4. Implement Script Version Control
5. Add Progressive Web App (PWA) Support
6. Optimize Bundle Size with Code Splitting
7. Implement Rate Limiting and Security Headers
8. Add Real-Time Collaboration Features
9. Implement Script Analytics Dashboard
10. Add Multi-Model AI Provider Support

---

1. Replace Babel with SWC for Faster Builds

Current State

The project uses `@vitejs/plugin-react` with Babel for JSX transformation, which is slower than modern alternatives.

Improvement

Replace Babel with SWC (Speedy Web Compiler) - a Rust-based compiler that offers **20-70x faster** compilation.

Implementation Details

```
# Install SWC plugin
npm uninstall @vitejs/plugin-react
npm install @vitejs/plugin-react-swc -D
```

```
// vite.config.ts - BEFORE
import react from '@vitejs/plugin-react';

// vite.config.ts - AFTER
import react from '@vitejs/plugin-react-swc';
```

Benefits

- **Faster HMR (Hot Module Replacement)**: Near-instant updates during development
- **Reduced Build Time**: 50-70% faster production builds
- **Lower Memory Usage**: Rust-based compilation is more memory efficient
- **Better Dev Experience**: Faster feedback loop encourages experimentation

Estimated Impact

Metric	Before	After
Dev Server Start	~3s	~1s
HMR Update	~500ms	~50ms
Production Build	~45s	~15s

Priority: HIGH

Effort: LOW (1-2 hours)

---

2. Implement Comprehensive Test Coverage

Current State

Only 11 tests exist (Button.test.tsx: 4, ScriptCard.test.tsx: 7). Critical paths like authentication, API calls, and AI workflows are untested.

Improvement

Achieve **80%+ code coverage** with unit, integration, and E2E tests following the testing pyramid.

Implementation Details

Testing Stack (2026 Best Practices)

```
# Unit & Integration Tests
npm install -D vitest @testing-library/react @testing-library/user-event j

# E2E Tests
npm install -D playwright @playwright/test

# API Mocking
npm install -D msw
```

Test Structure

```
src/
├── __tests__/
│   ├── unit/           # Pure function tests
│   ├── integration/    # Component + hook tests
│   └── e2e/            # Full user flow tests
├── __mocks__/          # Mock implementations
└── test/
    ├── setup.ts        # Test configuration
    └── utils.tsx       # Test utilities
```

Priority Test Coverage Areas

1. **Authentication Flow** (`AuthContext.tsx`)
2. Login/logout cycles
3. Token refresh handling
4. Session persistence
5. **API Services** (`services/*.ts`)
6. Error handling
7. Retry logic
8. Response parsing
9. **AI Workflows** (`api-simple.ts`, `assistantsApi.ts`)
10. Mock response handling
11. Real API integration
12. Streaming responses
13. **Script Management**
14. Upload/download
15. Edit/save
16. Delete confirmation

Example Test: AuthContext

```
// src/__tests__/integration/AuthContext.test.tsx
import { renderHook, act } from '@testing-library/react';
import { AuthProvider, useAuth } from '../../contexts/AuthContext';

describe('AuthContext', () => {
  it('should login user with valid credentials', async () => {
    const { result } = renderHook(() => useAuth(), {
      wrapper: AuthProvider,
    });

    await act(async () => {
      await result.current.login('user@test.com', 'password123');
    });

    expect(result.current.isAuthenticated).toBe(true);
    expect(result.current.user).toBeDefined();
  });

  it('should handle login failure gracefully', async () => {
    // Test error handling
  });

  it('should persist session across page reloads', async () => {
    // Test localStorage persistence
  });
});
```

Benefits

- **Catch Bugs Early**: Prevent regressions before they reach production
- **Confident Refactoring**: Change code without fear of breaking features
- **Documentation**: Tests serve as living documentation
- **CI/CD Integration**: Automated quality gates

Priority: CRITICAL

Effort: HIGH (2-3 weeks for 80% coverage)

---

3. Add Real AI Provider Integration

Current State

The system uses a mock response service when no API key is configured, providing generic responses.

Improvement

Implement production-ready AI integration with OpenAI, Anthropic Claude, and local models (Ollama).

Implementation Details

Backend Service Layer

```
// src/backend/src/services/ai/AIProviderFactory.ts
interface AIProvider {
  generateScript(prompt: string): Promise<string>;
  analyzeScript(script: string): Promise<ScriptAnalysis>;
  chat(messages: Message[]): AsyncGenerator<string>;
}

class OpenAIProvider implements AIProvider {
  private client: OpenAI;

  async generateScript(prompt: string): Promise<string> {
    const response = await this.client.chat.completions.create({
      model: 'gpt-4-turbo-2024-04-09',
      messages: [
        { role: 'system', content: POWERSHELL_SYSTEM_PROMPT },
        { role: 'user', content: prompt }
      ],
      temperature: 0.7,
    });
    return response.choices[0].message.content;
  }
}

class AnthropicProvider implements AIProvider {
  private client: Anthropic;

  async generateScript(prompt: string): Promise<string> {
    const response = await this.client.messages.create({
      model: 'claude-3-5-sonnet-20241022',
      max_tokens: 4096,
      messages: [{ role: 'user', content: prompt }],
      system: POWERSHELL_SYSTEM_PROMPT,
    });
    return response.content[0].text;
  }
```

```
}

// Factory pattern for provider selection
class AIProviderFactory {
  static create(provider: 'openai' | 'anthropic' | 'ollama'): AIProvider {
    switch (provider) {
      case 'openai': return new OpenAIProvider();
      case 'anthropic': return new AnthropicProvider();
      case 'ollama': return new OllamaProvider();
    }
  }
}
```

Streaming Response Support

```
// Enable real-time script generation feedback
async *streamGenerate(prompt: string): AsyncGenerator<string> {
  const stream = await this.client.chat.completions.create({
    model: 'gpt-4-turbo',
    messages: [...],
    stream: true,
  });

  for await (const chunk of stream) {
    yield chunk.choices[0]?.delta?.content || '';
  }
}
```

Frontend Streaming Integration

```
// src/frontend/src/hooks/useStreamingAI.ts
export function useStreamingAI() {
  const [output, setOutput] = useState('');
  const [isStreaming, setIsStreaming] = useState(false);

  const generate = async (prompt: string) => {
    setIsStreaming(true);
    setOutput('');

    const response = await fetch('/api/ai/generate/stream', {
      method: 'POST',
      body: JSON.stringify({ prompt }),
    });

    const reader = response.body?.getReader();
    const decoder = new TextDecoder();

    while (true) {
      const { done, value } = await reader!.read();
      if (done) break;
      setOutput(prev => prev + decoder.decode(value));
    }

    setIsStreaming(false);
  };

  return { output, isStreaming, generate };
}
```

Benefits

- **Production-Ready AI**: Real script analysis and generation
- **Provider Flexibility**: Switch between OpenAI/Anthropic/Ollama
- **Streaming UX**: Real-time feedback during generation
- **Cost Control**: Token tracking and usage limits

Priority: HIGH

Effort: MEDIUM (1-2 weeks)

---

4. Implement Script Version Control

Current State

Scripts are stored without version history. Edits overwrite the original with no way to revert.

Improvement

Implement Git-like version control for scripts with diff viewing, branching, and rollback capabilities.

Implementation Details

Database Schema

```
-- Script versions table
CREATE TABLE script_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  script_id UUID REFERENCES scripts(id) ON DELETE CASCADE,
  version_number INT NOT NULL,
  content TEXT NOT NULL,
  change_summary VARCHAR(500),
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW(),
  parent_version_id UUID REFERENCES script_versions(id),

  UNIQUE(script_id, version_number)
);

-- Index for fast version lookups
CREATE INDEX idx_script_versions_script_id ON script_versions(script_id);
CREATE INDEX idx_script_versions_created_at ON script_versions(created_at
```

Version Service

```
// src/backend/src/services/ScriptVersionService.ts
class ScriptVersionService {
  async createVersion(
    scriptId: string,
    content: string,
    changeSummary: string
  ): Promise<ScriptVersion> {
    const latestVersion = await this.getLatestVersion(scriptId);
    const newVersionNumber = (latestVersion?.version_number || 0) + 1;

    return await ScriptVersion.create({
      script_id: scriptId,
      version_number: newVersionNumber,
      content,
      change_summary: changeSummary,
      parent_version_id: latestVersion?.id,
    });
  }

  async getVersionHistory(scriptId: string): Promise<ScriptVersion[]> {
    return await ScriptVersion.findAll({
      where: { script_id: scriptId },
      order: [['version_number', 'DESC']],
    });
  }

  async getDiff(versionA: string, versionB: string): Promise<DiffResult> {
    const [a, b] = await Promise.all([
      ScriptVersion.findByPk(versionA),
      ScriptVersion.findByPk(versionB),
    ]);

    return diffLines(a.content, b.content);
  }

  async rollback(scriptId: string, targetVersion: number): Promise<Script>
    const version = await ScriptVersion.findOne({
```

```
      where: { script_id: scriptId, version_number: targetVersion },
    });

    // Create new version with rollback content
    await this.createVersion(
      scriptId,
      version.content,
      `Rollback to version ${targetVersion}`
    );

    return await Script.findByPk(scriptId);
  }
}
```

UI Component: Version Timeline

```
// src/frontend/src/components/VersionTimeline.tsx
function VersionTimeline({ scriptId }: Props) {
  const { versions, isLoading } = useScriptVersions(scriptId);
  const [selectedVersions, setSelectedVersions] = useState<string[]>([]);

  return (
    <div className="version-timeline">
      <h3>Version History</h3>
      {versions.map((version, index) => (
        <div key={version.id} className="version-item">
          <div className="version-dot" />
          <div className="version-content">
            <span className="version-number">v{version.version_number}</sp
            <span className="version-date">{formatDate(version.created_at)
            <p className="version-summary">{version.change_summary}</p>
            <div className="version-actions">
              <button onClick={() => viewVersion(version.id)}>View</button
              <button onClick={() => rollbackTo(version.version_number)}>
                Restore
              </button>
              <button onClick={() => toggleCompare(version.id)}>
                Compare
              </button>
            </div>
          </div>
        </div>
      ))}
    </div>
  );
}
```

Benefits

- **Safety Net**: Never lose work, always can rollback
- **Audit Trail**: Track who changed what and when
- **Collaboration**: Multiple people can work on scripts safely

- **Debugging**: Compare versions to find when bugs were introduced

Priority: HIGH

Effort: MEDIUM (1-2 weeks)

---

5. Add Progressive Web App (PWA) Support

Current State

The application requires an internet connection and cannot be installed as a standalone app.

Improvement

Convert to a PWA with offline support, push notifications, and installability.

Implementation Details

Vite PWA Plugin Setup

```
npm install -D vite-plugin-pwa
```

```
// vite.config.ts
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'robots.txt', 'apple-touch-icon.png']
      manifest: {
        name: 'PSScript - PowerShell Script Management',
        short_name: 'PSScript',
        description: 'AI-Powered PowerShell Script Management',
        theme_color: '#1a1a2e',
        background_color: '#1a1a2e',
        display: 'standalone',
        icons: [
          {
            src: 'pwa-192x192.png',
            sizes: '192x192',
            type: 'image/png',
          },
          {
            src: 'pwa-512x512.png',
            sizes: '512x512',
            type: 'image/png',
            purpose: 'any maskable',
          },
        ],
      },
      workbox: {
        globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}'],
        runtimeCaching: [
          {
            urlPattern: /^https:\/\/api\./i,
            handler: 'NetworkFirst',
            options: {
              cacheName: 'api-cache',
              expiration: {
```

```
                maxEntries: 100,
                maxAgeSeconds: 60 * 60 * 24, // 24 hours
              },
            },
          },
          {
            urlPattern: /\.(?:png|jpg|jpeg|svg|gif)$/,
            handler: 'CacheFirst',
            options: {
              cacheName: 'image-cache',
              expiration: {
                maxEntries: 50,
                maxAgeSeconds: 60 * 60 * 24 * 30, // 30 days
              },
            },
          },
        ],
      },
    }),
  ],
});
```

Offline Script Editor

```
// src/frontend/src/hooks/useOfflineScripts.ts
import { openDB } from 'idb';

const dbPromise = openDB('psscript-offline', 1, {
  upgrade(db) {
    db.createObjectStore('scripts', { keyPath: 'id' });
    db.createObjectStore('pending-sync', { keyPath: 'id', autoIncrement: t
  },
});

export function useOfflineScripts() {
  const saveOffline = async (script: Script) => {
    const db = await dbPromise;
    await db.put('scripts', script);
  };

  const getOfflineScripts = async (): Promise<Script[]> => {
    const db = await dbPromise;
    return db.getAll('scripts');
  };

  const queueForSync = async (action: SyncAction) => {
    const db = await dbPromise;
    await db.add('pending-sync', action);
  };

  const syncWhenOnline = async () => {
    const db = await dbPromise;
    const pending = await db.getAll('pending-sync');

    for (const action of pending) {
      try {
        await performSync(action);
        await db.delete('pending-sync', action.id);
      } catch (error) {
        console.error('Sync failed, will retry later:', error);
      }
```

```
    }
  };

  return { saveOffline, getOfflineScripts, queueForSync, syncWhenOnline };
}
```

Benefits

- **Offline Access**: View and edit scripts without internet
- **Installable**: Add to home screen on mobile/desktop
- **Fast Loading**: Cached assets load instantly
- **Push Notifications**: Alert users about script analysis results

Priority: MEDIUM

Effort: MEDIUM (1 week)

---

6. Optimize Bundle Size with Code Splitting

Current State

The entire application loads upfront, resulting in larger initial bundle size and slower first paint.

Improvement

Implement route-based code splitting and lazy loading for optimal performance.

Implementation Details

Route-Based Code Splitting

```
// src/frontend/src/App.tsx
import { lazy, Suspense } from 'react';
import { Routes, Route } from 'react-router-dom';

// Lazy load route components
const Dashboard = lazy(() => import('./pages/Dashboard'));
const ScriptManagement = lazy(() => import('./pages/ScriptManagement'));
const AIAssistant = lazy(() => import('./pages/AIAssistant'));
const Documentation = lazy(() => import('./pages/Documentation'));
const Settings = lazy(() => import('./pages/Settings'));

// Loading fallback
const PageLoader = () => (
  <div className="flex items-center justify-center h-screen">
    <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-
  </div>
);

function App() {
  return (
    <Suspense fallback={<PageLoader />}>
      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/scripts" element={<ScriptManagement />} />
        <Route path="/ai/assistant" element={<AIAssistant />} />
        <Route path="/documentation" element={<Documentation />} />
        <Route path="/settings/*" element={<Settings />} />
      </Routes>
    </Suspense>
  );
}
```

Manual Chunk Splitting

```
// vite.config.ts
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          // Vendor chunks
          'react-vendor': ['react', 'react-dom', 'react-router-dom'],
          'ui-vendor': ['@headlessui/react', '@heroicons/react'],
          'editor-vendor': ['monaco-editor', '@monaco-editor/react'],
          'chart-vendor': ['recharts', 'd3'],

          // Feature chunks
          'ai-features': [
            './src/pages/AIAssistant',
            './src/components/AIChat',
            './src/hooks/useAI',
          ],
        },
      },
    },
    // Target modern browsers only
    target: 'esnext',
    // Enable minification
    minify: 'esbuild',
    // Generate source maps for debugging
    sourcemap: true,
  },
});
```

Component-Level Lazy Loading

```
// Lazy load heavy components
const MonacoEditor = lazy(() => import('@monaco-editor/react'));
const MarkdownRenderer = lazy(() => import('./MarkdownRenderer'));
const ChartComponent = lazy(() => import('./ChartComponent'));

// Use with Suspense
function ScriptEditor({ script }: Props) {
  return (
    <Suspense fallback={<div>Loading editor...</div>}>
      <MonacoEditor
        language="powershell"
        value={script.content}
        theme="vs-dark"
      />
    </Suspense>
  );
}
```

Bundle Analysis

```
# Add bundle analyzer
npm install -D rollup-plugin-visualizer

# vite.config.ts
import { visualizer } from 'rollup-plugin-visualizer';

plugins: [
  visualizer({
    filename: 'dist/stats.html',
    open: true,
    gzipSize: true,
  }),
]
```

Target Metrics

Metric	Current	Target
Initial Bundle	~800KB	<200KB
First Contentful Paint	~2.5s	<1.5s
Time to Interactive	~4s	<2.5s
Lighthouse Score	~70	>90

Priority: HIGH

Effort: LOW (2-3 days)

---

7. Implement Rate Limiting and Security Headers

Current State

No rate limiting on API endpoints, missing security headers, potential for abuse.

Improvement

Add comprehensive security measures including rate limiting, CORS hardening, and security headers.

Implementation Details

Rate Limiting with Redis

```
// src/backend/src/middleware/rateLimiter.ts
import rateLimit from 'express-rate-limit';
import RedisStore from 'rate-limit-redis';
import { redisClient } from '../config/redis';

// General API rate limit
export const apiLimiter = rateLimit({
  store: new RedisStore({
    sendCommand: (...args: string[]) => redisClient.sendCommand(args),
  }),
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // 100 requests per window
  message: {
    error: 'Too many requests, please try again later.',
    retryAfter: 15 * 60,
  },
  standardHeaders: true,
  legacyHeaders: false,
});

// Stricter limit for AI endpoints (expensive operations)
export const aiLimiter = rateLimit({
  store: new RedisStore({
    sendCommand: (...args: string[]) => redisClient.sendCommand(args),
  }),
  windowMs: 60 * 1000, // 1 minute
  max: 10, // 10 AI requests per minute
  message: {
    error: 'AI rate limit exceeded. Please wait before generating more scr
  },
});

// Auth endpoints (prevent brute force)
export const authLimiter = rateLimit({
  store: new RedisStore({
```

```
    sendCommand: (...args: string[]) => redisClient.sendCommand(args),
  }),
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 login attempts per window
  skipSuccessfulRequests: true, // Don't count successful logins
});
```

Security Headers with Helmet

```
// src/backend/src/middleware/security.ts
import helmet from 'helmet';

export const securityMiddleware = helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "'unsafe-inline'"], // Required for Monaco edi
      styleSrc: ["'self'", "'unsafe-inline'", 'https://fonts.googleapis.co
      fontSrc: ["'self'", 'https://fonts.gstatic.com'],
      imgSrc: ["'self'", 'data:', 'https:'],
      connectSrc: ["'self'", 'https://api.openai.com', 'https://api.anthro
      frameSrc: ["'none'"],
      objectSrc: ["'none'"],
    },
  },
  crossOriginEmbedderPolicy: false, // Required for some features
  crossOriginOpenerPolicy: { policy: 'same-origin' },
  crossOriginResourcePolicy: { policy: 'same-site' },
  dnsPrefetchControl: { allow: false },
  frameguard: { action: 'deny' },
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true,
  },
  ieNoOpen: true,
  noSniff: true,
  originAgentCluster: true,
  permittedCrossDomainPolicies: { permittedPolicies: 'none' },
  referrerPolicy: { policy: 'strict-origin-when-cross-origin' },
  xssFilter: true,
});
```

Input Validation with Zod

```
// src/backend/src/validators/scriptValidators.ts
import { z } from 'zod';

export const createScriptSchema = z.object({
  title: z.string()
    .min(1, 'Title is required')
    .max(200, 'Title must be less than 200 characters')
    .regex(/^[^<>]*$/, 'Title contains invalid characters'),

  content: z.string()
    .min(1, 'Content is required')
    .max(1000000, 'Script is too large (max 1MB)'),

  description: z.string()
    .max(5000, 'Description must be less than 5000 characters')
    .optional(),

  tags: z.array(z.string().max(50)).max(20).optional(),
});

export const generateScriptSchema = z.object({
  prompt: z.string()
    .min(10, 'Prompt must be at least 10 characters')
    .max(5000, 'Prompt must be less than 5000 characters'),

  options: z.object({
    includeComments: z.boolean().default(true),
    includeErrorHandling: z.boolean().default(true),
    targetPSVersion: z.enum(['5.1', '7.0', '7.4']).default('7.4'),
  }).optional(),
});
```

Benefits

- **DDoS Protection**: Rate limiting prevents abuse
- **XSS Prevention**: CSP headers block malicious scripts

- **Data Integrity**: Input validation prevents injection attacks
- **Compliance**: Meet security audit requirements

Priority: CRITICAL

Effort: MEDIUM (1 week)

---

8. Add Real-Time Collaboration Features

Current State

Single-user editing only. No way for teams to collaborate on scripts in real-time.

Improvement

Implement WebSocket-based real-time collaboration with presence indicators and conflict resolution.

Implementation Details

WebSocket Server Setup

```
// src/backend/src/websocket/collaborationServer.ts
import { Server } from 'socket.io';
import { createAdapter } from '@socket.io/redis-adapter';

export function setupCollaborationServer(httpServer: HttpServer) {
  const io = new Server(httpServer, {
    cors: {
      origin: process.env.FRONTEND_URL,
      credentials: true,
    },
    adapter: createAdapter(pubClient, subClient), // Redis for scaling
  });

  io.on('connection', (socket) => {
    socket.on('join-script', async (scriptId: string) => {
      socket.join(`script:${scriptId}`);

      // Notify others of presence
      const users = await getActiveUsers(scriptId);
      io.to(`script:${scriptId}`).emit('presence-update', users);
    });

    socket.on('edit', async (data: EditOperation) => {
      // Apply operational transformation
      const transformed = applyOT(data);

      // Broadcast to others in the room
      socket.to(`script:${data.scriptId}`).emit('remote-edit', transformed

      // Persist change
      await saveEdit(transformed);
    });

    socket.on('cursor-move', (data: CursorPosition) => {
      socket.to(`script:${data.scriptId}`).emit('remote-cursor', {
```

```
        userId: socket.userId,
        position: data.position,
      });
    });

    socket.on('disconnect', async () => {
      // Update presence for all scripts user was in
      await removeUserPresence(socket.userId);
    });
  });

  return io;
}
```

Operational Transformation for Conflict Resolution

```
// src/backend/src/websocket/operationalTransform.ts
interface Operation {
  type: 'insert' | 'delete' | 'retain';
  position: number;
  content?: string;
  length?: number;
  timestamp: number;
}

export function transformOperations(
  localOp: Operation,
  remoteOp: Operation
): [Operation, Operation] {
  // Transform operations to maintain consistency
  if (localOp.type === 'insert' && remoteOp.type === 'insert') {
    if (localOp.position <= remoteOp.position) {
      return [
        localOp,
        { ...remoteOp, position: remoteOp.position + localOp.content!.leng
      ];
    } else {
      return [
        { ...localOp, position: localOp.position + remoteOp.content!.lengt
        remoteOp,
      ];
    }
  }
  // ... handle other cases
}
```

Frontend Collaboration Hook

```
// src/frontend/src/hooks/useCollaboration.ts
export function useCollaboration(scriptId: string) {
  const [collaborators, setCollaborators] = useState<Collaborator[]>([]);
  const [cursors, setCursors] = useState<Map<string, CursorPosition>>();
  const socketRef = useRef<Socket>();

  useEffect(() => {
    const socket = io(WS_URL, { auth: { token: getAuthToken() } });
    socketRef.current = socket;

    socket.emit('join-script', scriptId);

    socket.on('presence-update', setCollaborators);
    socket.on('remote-cursor', (data) => {
      setCursors(prev => new Map(prev).set(data.userId, data.position));
    });
    socket.on('remote-edit', (operation) => {
      applyRemoteOperation(operation);
    });

    return () => {
      socket.disconnect();
    };
  }, [scriptId]);

  const sendEdit = (operation: Operation) => {
    socketRef.current?.emit('edit', { scriptId, ...operation });
  };

  const sendCursorMove = (position: CursorPosition) => {
    socketRef.current?.emit('cursor-move', { scriptId, position });
  };

  return { collaborators, cursors, sendEdit, sendCursorMove };
}
```

Benefits

- **Team Productivity**: Multiple people can edit simultaneously
- **Real-Time Feedback**: See changes as they happen
- **Conflict Prevention**: OT ensures consistent state
- **Awareness**: Know who's working on what

Priority: MEDIUM

Effort: HIGH (3-4 weeks)

---

9. Implement Script Analytics Dashboard

Current State

No visibility into script usage, performance metrics, or user behavior patterns.

Improvement

Add comprehensive analytics tracking with visualization dashboard.

Implementation Details

Analytics Event Tracking

```
// src/backend/src/services/AnalyticsService.ts
interface AnalyticsEvent {
  eventType: string;
  scriptId?: string;
  userId: string;
  metadata: Record<string, any>;
  timestamp: Date;
}

class AnalyticsService {
  async trackEvent(event: AnalyticsEvent): Promise<void> {
    // Store in TimescaleDB for time-series data
    await db.query(`
      INSERT INTO analytics_events (event_type, script_id, user_id, metada
      VALUES ($1, $2, $3, $4, $5)
    `, [event.eventType, event.scriptId, event.userId, event.metadata, eve
  }

  async getScriptMetrics(scriptId: string, timeRange: TimeRange): Promise<
    const [views, downloads, executions, aiAnalyses] = await Promise.all([
      this.countEvents('script_view', scriptId, timeRange),
      this.countEvents('script_download', scriptId, timeRange),
      this.countEvents('script_execution', scriptId, timeRange),
      this.countEvents('ai_analysis', scriptId, timeRange),
    ]);

    return { views, downloads, executions, aiAnalyses };
  }

  async getDashboardMetrics(userId: string): Promise<DashboardMetrics> {
    return {
      totalScripts: await this.countUserScripts(userId),
      totalViews: await this.sumUserScriptViews(userId),
      aiTokensUsed: await this.sumAITokenUsage(userId),
      topScripts: await this.getTopScripts(userId, 5),
```

```
      activityHeatmap: await this.getActivityHeatmap(userId),
      trendData: await this.getTrendData(userId, '30d'),
    };
  }
}
```

Dashboard UI Component

```
// src/frontend/src/pages/Analytics.tsx
function AnalyticsDashboard() {
  const { metrics, isLoading } = useAnalytics();

  return (
    <div className="analytics-dashboard">
      {/* Summary Cards */}
      <div className="grid grid-cols-4 gap-4">
        <MetricCard
          title="Total Scripts"
          value={metrics.totalScripts}
          change={metrics.scriptsChange}
          icon={<DocumentIcon />}
        />
        <MetricCard
          title="Total Views"
          value={metrics.totalViews}
          change={metrics.viewsChange}
          icon={<EyeIcon />}
        />
        <MetricCard
          title="AI Generations"
          value={metrics.aiGenerations}
          change={metrics.generationsChange}
          icon={<SparklesIcon />}
        />
        <MetricCard
          title="Team Members"
          value={metrics.teamMembers}
          icon={<UsersIcon />}
        />
      </div>

      {/* Charts */}
      <div className="grid grid-cols-2 gap-6 mt-6">
        <Card title="Script Activity">
          <LineChart data={metrics.trendData} />
```

```
          </Card>
          <Card title="Top Scripts">
            <BarChart data={metrics.topScripts} />
          </Card>
          <Card title="Activity Heatmap">
            <CalendarHeatmap data={metrics.activityHeatmap} />
          </Card>
          <Card title="AI Usage">
            <PieChart data={metrics.aiUsageByType} />
          </Card>
        </div>

        {/* Recent Activity Feed */}
        <Card title="Recent Activity" className="mt-6">
          <ActivityFeed events={metrics.recentEvents} />
        </Card>
      </div>
    );
  }
```

Benefits

- **Data-Driven Decisions**: Understand what scripts are most valuable
- **Usage Patterns**: Identify peak usage times and optimize
- **Cost Management**: Track AI token consumption
- **Team Insights**: See how the team uses the platform

Priority: MEDIUM

Effort: MEDIUM (1-2 weeks)

---

10. Add Multi-Model AI Provider Support

Current State

Limited to single AI provider, no model selection, no cost comparison.

Improvement

Support multiple AI providers with model selection, cost tracking, and intelligent routing.

Implementation Details

Provider Configuration

```
// src/backend/src/config/aiProviders.ts
export const AI_PROVIDERS = {
  openai: {
    models: [
      { id: 'gpt-4-turbo', name: 'GPT-4 Turbo', costPer1kTokens: 0.01 },
      { id: 'gpt-4o', name: 'GPT-4o', costPer1kTokens: 0.005 },
      { id: 'gpt-3.5-turbo', name: 'GPT-3.5 Turbo', costPer1kTokens: 0.000
    ],
    capabilities: ['generation', 'analysis', 'chat'],
  },
  anthropic: {
    models: [
      { id: 'claude-3-5-sonnet-20241022', name: 'Claude 3.5 Sonnet', costP
      { id: 'claude-3-opus-20240229', name: 'Claude 3 Opus', costPer1kToke
      { id: 'claude-3-haiku-20240307', name: 'Claude 3 Haiku', costPer1kTo
    ],
    capabilities: ['generation', 'analysis', 'chat'],
  },
  ollama: {
    models: [
      { id: 'codellama:34b', name: 'CodeLlama 34B', costPer1kTokens: 0 },
      { id: 'deepseek-coder:33b', name: 'DeepSeek Coder', costPer1kTokens:
    ],
    capabilities: ['generation', 'analysis'],
    local: true,
  },
};
```

Intelligent Model Router

```
// src/backend/src/services/ai/ModelRouter.ts
class ModelRouter {
  async selectOptimalModel(task: AITask): Promise<ModelSelection> {
    const { taskType, complexity, budget, latencyRequirement } = task;

    // Score each model based on criteria
    const scores = this.availableModels.map(model => ({
      model,
      score: this.calculateScore(model, {
        taskType,
        complexity,
        budget,
        latencyRequirement,
      }),
    }));

    // Sort by score and return best match
    scores.sort((a, b) => b.score - a.score);
    return scores[0].model;
  }

  private calculateScore(model: AIModel, criteria: SelectionCriteria): num
    let score = 0;

    // Task capability match
    if (model.capabilities.includes(criteria.taskType)) score += 30;

    // Cost efficiency (normalized)
    const costScore = (1 - model.costPer1kTokens / this.maxCost) * 25;
    score += Math.max(0, costScore);

    // Quality rating for task type
    score += model.qualityRatings[criteria.taskType] * 25;

    // Latency match
    if (model.avgLatency <= criteria.latencyRequirement) score += 20;
```

```
    return score;
  }
}
```

Settings UI for Model Selection

```
// src/frontend/src/pages/Settings/AISettings.tsx
function AISettings() {
  const { providers, selectedModel, setSelectedModel } = useAISettings();

  return (
    <div className="ai-settings">
      <h2>AI Provider Settings</h2>

      {/* Provider Selection */}
      <section>
        <h3>Available Providers</h3>
        <div className="grid grid-cols-3 gap-4">
          {Object.entries(providers).map(([key, provider]) => (
            <ProviderCard
              key={key}
              provider={provider}
              isConfigured={provider.isConfigured}
              onConfigure={() => openConfigModal(key)}
            />
          ))}
        </div>
      </section>

      {/* Model Selection */}
      <section className="mt-8">
        <h3>Default Model</h3>
        <ModelSelector
          models={getAllModels()}
          selected={selectedModel}
          onChange={setSelectedModel}
          showCostComparison
        />
      </section>

      {/* Auto-Routing */}
      <section className="mt-8">
        <h3>Intelligent Routing</h3>
```

```
          <Toggle
            label="Auto-select best model for each task"
            description="Uses task complexity and budget to choose optimal m
          />
          <Slider
            label="Budget Priority"
            min={0}
            max={100}
            description="0 = Quality first, 100 = Cost first"
          />
        </section>

        {/* Usage & Costs */}
        <section className="mt-8">
          <h3>Usage This Month</h3>
          <UsageChart data={monthlyUsage} />
          <CostBreakdown costs={monthlyCosts} />
        </section>
      </div>
    );
  }
```

Benefits

- **Flexibility**: Choose the right model for each task
- **Cost Optimization**: Use cheaper models for simple tasks
- **Redundancy**: Fallback to other providers if one fails
- **Future-Proof**: Easily add new providers as they emerge

Priority: HIGH

Effort: HIGH (2-3 weeks)

Implementation Priority Matrix

Improvement	Priority	Effort	Impact	Recommended Order
SWC Migration	HIGH	LOW	HIGH	1
Security Headers	CRITICAL	MEDIUM	HIGH	2
Bundle Optimization	HIGH	LOW	HIGH	3
Test Coverage	CRITICAL	HIGH	HIGH	4
Real AI Integration	HIGH	MEDIUM	HIGH	5
Multi-Model Support	HIGH	HIGH	MEDIUM	6
Script Versioning	HIGH	MEDIUM	HIGH	7
PWA Support	MEDIUM	MEDIUM	MEDIUM	8
Analytics Dashboard	MEDIUM	MEDIUM	MEDIUM	9
Real-Time Collab	MEDIUM	HIGH	LOW	10

Quick Wins (Can Be Done This Week)

1. **SWC Migration** - 1-2 hours, immediate build speed improvement
2. **Bundle Optimization** - 2-3 days, significant performance gain
3. **Security Headers** - 1 day, critical security improvement

---

References

- Vite Official Performance Guide
- FastAPI Best Practices
- React TypeScript Best Practices 2026
- FastAPI Production Deployment

Generated: 2026-01-12 Last Updated: 2026-01-12

Generated 2026-04-29 14:08 UTC